

```

import matplotlib.pyplot as plt
import numpy as np

# -----
# Método de Gauss-Jordan manual
# -----
def gauss_jordan(A, b):
    n = len(b)
    M = np.hstack([A.astype(float), b.reshape(-1,1)])

    for i in range(n):
        # Pivô
        pivot = M[i][i]
        if pivot == 0:
            raise ValueError("Pivô nulo encontrado, sistema singular.")
        M[i] = M[i] / pivot

        # Eliminação
        for j in range(n):
            if j != i:
                M[j] = M[j] - M[j][i] * M[i]

    return M[:, -1]

# -----
# Montagem da matriz de equilíbrio
# -----
def montar_equilibrio(N, coords, barras, cargas, apoios):
    eqs = 2 * N
    B = len(barras)
    A = np.zeros((eqs, B + sum(apoios)))
    b = np.zeros(eqs)

    # Forças externas
    for i in range(N):
        b[2*i] = cargas[i][0]
        b[2*i+1] = cargas[i][1]

    # Barras
    for k, (i, j) in enumerate(barras):
        xi, yi = coords[i]
        xj, yj = coords[j]
        L = np.sqrt((xj-xi)**2 + (yj-yi)**2)

```

```
cx = (xj-xi)/L
cy = (yj-yi)/L
```

```
A[2*i][k] = cx
A[2*i+1][k] = cy
A[2*j][k] = -cx
A[2*j+1][k] = -cy
```

```
# Apoios
col = B
for i, tipo in enumerate(apoios):
    if tipo == 1: # móvel (restrição em y)
        A[2*i+1][col] = 1
        col += 1
    elif tipo == 2: # fixo (restrição em x e y)
        A[2*i][col] = 1
        col += 1
        A[2*i+1][col] = 1
        col += 1

return A, b
```

```
# -----
# Classificação das barras
# -----
def classificar_barras(forcas):
    return ["Tração" if f > 0 else "Compressão" if f < 0 else "Nula" for f in forcas]
```

```
# -----
# Visualizações gráficas
# -----
def plotar_resultados(coords, barras, forcas):
    # Treliza original
    plt.figure()
    for (i,j) in barras:
        plt.plot([coords[i][0], coords[j][0]], [coords[i][1], coords[j][1]], 'k-')
    for i,(x,y) in enumerate(coords):
        plt.text(x,y,f"{i}", color="red")
    plt.title("Treliza Original")
    plt.axis("equal")
    plt.show()
```

```
# Mapa de calor
```

```

plt.figure()
for k,(i,j) in enumerate(barras):
    cor = 'blue' if forcas[k]>0 else 'red' if forcas[k]<0 else 'gray'
    plt.plot([coords[i][0], coords[j][0]], [coords[i][1], coords[j][1]], color=cor, linewidth=2)
plt.title("Mapa de Forças (Tração=Azul, Compressão=Vermelho)")
plt.axis("equal")
plt.show()

```

Gráfico de barras

```

plt.figure()
plt.bar(range(len(forcas)), np.abs(forcas))
plt.title("Magnitude das Forças em Cada Barra")
plt.xlabel("Barra")
plt.ylabel("Força")
plt.show()

```

Histograma

```

plt.figure()
plt.hist(forcas, bins=10, color="gray", edgecolor="black")
plt.title("Distribuição das Forças Internas")
plt.xlabel("Força")
plt.ylabel("Frequência")
plt.show()

```

Exemplo: Treliza Warren (teste)

if __name__ == "__main__":

 N = 4

 coords = [(0,0),(2,0),(1,1),(3,1)]

 barras = [(0,1),(0,2),(1,2),(1,3),(2,3)]

 cargas = [(0,0),(0,0),(0,-1000),(0,0)]

 apoios = [2,1,0,0] # nó 0 fixo, nó 1 móvel

 A, b = montar_equilibrio(N, coords, barras, cargas, apoios)

 x = gauss_jordan(A, b)

 forcas = x[:len(barras)]

 classificacao = classificar_barras(forcas)

 print("Forças nas barras:", forcas)

 print("Classificação:", classificacao)

 plotar_resultados(coords, barras, forcas)

